```
import { createServerFn } from "@tanstack/react-start";
import { getRequest } from "@tanstack/react-start/server";
import type { SupabaseClient } from "@supabase/supabase-js";
import { z } from "zod";

import type { Database } from "@/integrations/supabase/types";
import {
  ACTIVE_RESERVATION_STATUSES,
  buildPublicAvailability,
  evaluateCrewAvailability,
  isActiveReservationStatus,
  pickLeastBusyCrew,
  planContinuationSegments as planContinuationSegmentsFromAvailability,
  type Assignment,
} from "./availability";
import {
  addDays,
  buildBookingTimeSlots,
  DEFAULT_BOOKING_HOURS,
  decodeBlockReason,
  earliestBookableDate,
  formatTime,
  NAME_PART_PATTERN,
  NAME_PART_VALIDATION_MESSAGE,
  bookingDurationForFirstDay,
  bookingDurationOverflowMinutes,
  isBookingStartTime,
  isBookingTimeRangeWithinHours,
  isValidCalendarDate,
  isShopOpenDate,
  isSlotBookable,
  manilaNow,
  MAX_BOOKING_SERVICE_SELECTIONS,
  normalizePhilippineMobile,
  phoneSchema,
  REFERENCE_CODE_PATTERN,
  normalizeReferenceCode,
  SHOP,
  timeToMinutes,
} from "./shop";
import type { AppointmentAction } from "./appointment-verification.server";

type BookingRules = {
  operatingHours: {
    openingTime: string;
```

```
    closingTime: string;
  };
  minimumBookingLeadHours: number;
  maxAdvanceBookingDays: number;
  allowSameDayAppointments: boolean;
  cancellationNoticeHours: number;
  reschedulingNoticeHours: number;
};

const defaultBookingRules: BookingRules = {
  operatingHours: DEFAULT_BOOKING_HOURS,
  minimumBookingLeadHours: 48,
  maxAdvanceBookingDays: 30,
  allowSameDayAppointments: false,
  cancellationNoticeHours: 48,
  reschedulingNoticeHours: 48,
};

async function getBookingRules(): Promise<BookingRules> {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const { data, error } = await supabaseAdmin
      .from("shop_settings")
      .select(
        "opening_time,closing_time,minimum_booking_lead_hours,max_advance_b
ooking_days,allow_same_day_appointments,cancellation_notice_hours,resched
uling_notice_hours",
      )
      .eq("id", true)
      .maybeSingle();
    if (error || !data) {
      if (error) console.warn("[Booking settings] using defaults", error.message);
      return defaultBookingRules;
    }
    return {
      operatingHours: {
        openingTime: String(data.opening_time).slice(0, 5),
        closingTime: String(data.closing_time).slice(0, 5),
      },
      minimumBookingLeadHours: data.minimum_booking_lead_hours,
      maxAdvanceBookingDays: data.max_advance_booking_days,
      allowSameDayAppointments: data.allow_same_day_appointments,
      cancellationNoticeHours: data.cancellation_notice_hours,
      reschedulingNoticeHours: data.rescheduling_notice_hours,
```

```
    };
  } catch (error) {
    console.warn("[Booking settings] using defaults", error);
    return defaultBookingRules;
  }
}

function firstBookableDate(rules: BookingRules, minimumLeadHours =
rules.minimumBookingLeadHours) {
  let date = earliestBookableDate(minimumLeadHours);
  if (!rules.allowSameDayAppointments && date === manilaNow().date) date =
addDays(date, 1);
  return date;
}

const currentYear = new Date().getFullYear();

const motorcycleSelectionSchema = z.object({
  brand: z.string().trim().min(1).max(50),
  model: z.string().trim().min(1).max(50),
});

type ResolvedService = {
  id: string;
  name: string;
  price: number;
  durationMinutes: number;
  pricingSource: "small_bike" | "large_bike";
};

type MotorcycleCcCategory = "small_bike" | "big_bike";

function resolveServicePricing(
  services: Array<{
    id: string;
    name: string;
    price: number;
    duration_minutes: number | null;
    large_bike_price: number | null;
    large_bike_duration_minutes: number | null;
  }>,
  motorcycleCategory?: MotorcycleCcCategory,
): { services: ResolvedService[] } | { error: string } {
  if (motorcycleCategory === "big_bike") {
    const missingConfiguration = services.find(
```

```
      (service) =>
        service.large_bike_price === null || service.large_bike_duration_minutes
=== null,
    );
    if (missingConfiguration) {
      return {
        error: `${missingConfiguration.name} does not have a Large Bike price and
duration configured yet.`,
      };
    }
    return {
      services: services.map((service) => ({
        id: service.id,
        name: service.name,
        price: Number(service.large_bike_price),
        durationMinutes: service.large_bike_duration_minutes ?? 60,
        pricingSource: "large_bike",
      })),
    };
  }

  return {
    services: services.map((service) => ({
      id: service.id,
      name: service.name,
      price: Number(service.price),
      durationMinutes: service.duration_minutes ?? 60,
      pricingSource: "small_bike",
    })),
  };
}

const availabilitySchema = z
  .object({
    days: z.number().int().min(7).max(90).default(45),
    serviceIds: z
      .array(z.string().uuid())
      .max(MAX_BOOKING_SERVICE_SELECTIONS)
      .refine((ids) => new Set(ids).size === ids.length, "Services must be unique")
      .default([]),
    // Administrators editing an appointment need to see the current slot as
    // available while still applying the exact same booking rules to every
    // other appointment.
    excludeAppointmentId: z.string().uuid().optional(),
    // Admin edits may refer to a motorcycle that has since been deactivated.
```

```
    // Its catalog classification still determines the saved service snapshot.
    allowInactiveMotorcycle: z.boolean().default(false),
    allowMultiDayContinuation: z.boolean().default(false),
    rescheduling: z.boolean().default(false),
    motorcycle: motorcycleSelectionSchema.optional(),
    rescheduleReference: z
      .string()
      .trim()
      .regex(REFERENCE_CODE_PATTERN, "Enter a valid original appointment
reference.")
      .transform(normalizeReferenceCode)
      .optional(),
    reschedulePhone: phoneSchema.optional(),
  })
  .superRefine((data, context) => {
    if (Boolean(data.rescheduleReference) !== Boolean(data.reschedulePhone)) {
      context.addIssue({
        code: z.ZodIssueCode.custom,
        message: "A reschedule reference and mobile number must be provided
together.",
      });
    }
  });

const namePartSchema = z
  .string()
  .trim()
  .max(40)
  .regex(NAME_PART_PATTERN, NAME_PART_VALIDATION_MESSAGE);

const emailSchema = z
  .string()
  .trim()
  .email("Enter a valid email address.")
  .max(254)
  .transform((email) => email.toLocaleLowerCase());

const bookingSchema = z
  .object({
    firstName: namePartSchema.default(""),
    middleName: namePartSchema.default(""),
    lastName: namePartSchema.default(""),
    phone: phoneSchema,
    email: emailSchema.optional(),
    motoBrand: z.string().trim().min(1).max(50),
```

```
    motoModel: z.string().trim().min(1).max(50),
    motoVariant: z.string().trim().max(50).optional().or(z.literal("")),
    motoYear: z.number().int().min(1970).max(currentYear),
    plateNumber: z.string().trim().min(2).max(20),
    serviceIds: z
      .array(z.string().uuid())
      .min(1)
      .max(MAX_BOOKING_SERVICE_SELECTIONS)
      .refine((ids) => new Set(ids).size === ids.length, "Services must be unique"),
    date: z.string().refine(isValidCalendarDate, "Enter a valid appointment date."),
    startTime: z
      .string()
      .regex(/^\d{2}:\d{2}$/, "Enter a valid appointment start time.")
      .refine(
        (value) => Number.isFinite(timeToMinutes(value)),
        "Enter a valid appointment start time.",
      ),
    notes: z.string().trim().max(500).optional().or(z.literal("")),
    wantsProducts: z.boolean(),
    turnstileToken: z.string().trim().max(2048).default(""),
    idempotencyKey: z.string().uuid(),
    termsAccepted: z.literal(true),
    multiDayContinuationAccepted: z.boolean().default(false),
    rescheduleReference: z
      .string()
      .trim()
      .regex(REFERENCE_CODE_PATTERN, "Enter a valid original appointment
reference.")
      .optional(),
    rescheduleReason: z.string().trim().max(500).optional().or(z.literal("")),
    // Issued by verifyAppointmentActionCode; required whenever
rescheduleReference is set.
    verificationToken: z.string().trim().max(100).optional(),
  })
  .superRefine((data, context) => {
    if (!data.rescheduleReference && !data.email) {
      context.addIssue({
        code: z.ZodIssueCode.custom,
        path: ["email"],
        message: "Enter a valid email address.",
      });
    }
    if (data.rescheduleReference && !data.rescheduleReason?.trim()) {
      context.addIssue({
        code: z.ZodIssueCode.custom,
```

```
        path: ["rescheduleReason"],
        message: "A reason for rescheduling is required.",
      });
    }
  });

function fullName(firstName: string, middleName: string, lastName: string) {
  return [firstName.trim(), middleName.trim(),
lastName.trim()].filter(Boolean).join(" ");
}

function unavailableAvailability(from: string, to: string, error: string) {
  return {
    from,
    to,
    error,
    totalDurationMinutes: 60,
    dates: [],
    fullyBookedDates: [],
    slotsByDate: {},
  };
}

type ContinuationSegment = {
  segmentNumber: number;
  appointmentDate: string;
  startTime: string;
  bookingDurationMinutes: number;
  assignedCrewId: string;
};

/**
 * A pending reschedule request holds its requested time until the shop reviews
 * it, so other customers cannot take it in the meantime.
 */
function rescheduleHoldsQuery(supabaseAdmin: SupabaseClient<Database>,
from: string, to: string) {
  return supabaseAdmin
    .from("appointments")
    .select("id,pending_reschedule_date,pending_reschedule_start_time,booking_
duration_minutes")
    .not("pending_reschedule_request_id", "is", null)
    .eq("is_archived", false)
    .is("rescheduled_to_appointment_id", null)
    .in("status", ["pending", "confirmed"])
```

```
    .gte("pending_reschedule_date", from)
    .lte("pending_reschedule_date", to);
}

/** Holds count as reservations that do not have a mechanic yet. */
function rescheduleHoldAssignments(
  rows: Array<{
    id: string;
    pending_reschedule_date: string | null;
    pending_reschedule_start_time: string | null;
    booking_duration_minutes: number | null;
  }>,
  excludedAppointmentId?: string | null,
): Assignment[] {
  return rows.flatMap((row) =>
    row.id !== excludedAppointmentId &&
    row.pending_reschedule_date &&
    row.pending_reschedule_start_time
      ? [
          {
            date: row.pending_reschedule_date,
            startTime: String(row.pending_reschedule_start_time).slice(0, 5),
            // The same default the database uses for a hold.
            durationMinutes: row.booking_duration_minutes ?? 75,
            crewId: null,
          },
        ]
      : [],
  );
}

/** Most appointments one mobile number or email address can have coming up
at once. */
const MAX_UPCOMING_APPOINTMENTS = 3;

/**
 * The reason a new booking is refused for having too many upcoming
 * appointments, or null. The database enforces the same limit.
 */
async function upcomingAppointmentLimitError(
  supabaseAdmin: SupabaseClient<Database>,
  phone: string,
  email: string | undefined,
): Promise<string | null> {
  const upcoming = () =>
```

```
    supabaseAdmin
      .from("appointments")
      .select("id", { count: "exact", head: true })
      .eq("is_archived", false)
      .is("rescheduled_to_appointment_id", null)
      .in("status", ["pending", "confirmed", "in_progress"])
      .gte("appointment_date", manilaNow().date);
  const normalizedEmail = email?.trim().toLowerCase();
  const [byPhone, byEmail] = await Promise.all([
    upcoming().eq("phone", phone),
    normalizedEmail ? upcoming().eq("email", normalizedEmail) :
Promise.resolve(null),
  ]);
  if (byPhone.error || byEmail?.error) {
    console.error("[Booking] upcoming appointment count failed", byPhone.error
?? byEmail?.error);
    return null; // The database limit still applies when the booking is saved.
  }
  if ((byPhone.count ?? 0) >= MAX_UPCOMING_APPOINTMENTS) {
    return upcomingLimitMessage("mobile number");
  }
  if ((byEmail?.count ?? 0) >= MAX_UPCOMING_APPOINTMENTS) {
    return upcomingLimitMessage("email address");
  }
  return null;
}

function upcomingLimitMessage(contact: "mobile number" | "email address") {
  return `This ${contact} already has ${MAX_UPCOMING_APPOINTMENTS}
upcoming appointments, the most allowed at one time. Please cancel one or wait
until one is done before booking another.`;
}

async function planContinuationSegments({
  supabaseAdmin,
  firstDate,
  remainingMinutes,
  lastAvailableDate,
  operatingHours,
  slotConfig,
}: {
  supabaseAdmin: SupabaseClient<Database>;
  firstDate: string;
  remainingMinutes: number;
  lastAvailableDate: string;
```

```
  operatingHours: BookingRules["operatingHours"];
  slotConfig: Array<{ startTime: string; capacity: number }>;
}): Promise<{ ok: true; segments: ContinuationSegment[] } | { ok: false; error:
string }> {
  const from = addDays(firstDate, 1);
  if (from > lastAvailableDate) {
    return {
      ok: false,
      error: "There is not enough future shop time available to complete these
services.",
    };
  }

  const [
    blocksRes,
    schedulesRes,
    crewRes,
    exceptionsRes,
    appointmentsRes,
    continuationsRes,
    holdsRes,
  ] = await Promise.all([
    supabaseAdmin
      .from("schedule_blocks")
      .select("block_date,start_time,reason")
      .eq("is_active", true)
      .gte("block_date", from)
      .lte("block_date", lastAvailableDate),
    supabaseAdmin
      .from("crew_schedules")
      .select("crew_id,schedule_date,start_time,end_time,is_working")
      .gte("schedule_date", from)
      .lte("schedule_date", lastAvailableDate),
    supabaseAdmin.from("crew_members").select("id").eq("is_active",
true).eq("is_archived", false),
    supabaseAdmin
      .from("crew_availability_exceptions")
      .select("crew_id,start_date,end_date,start_time,end_time,is_all_day")
      .lte("start_date", lastAvailableDate)
      .gte("end_date", from),
    supabaseAdmin
      .from("appointments")
      .select("appointment_date,start_time,assigned_crew_id,booking_duration_mi
nutes")
      .eq("is_archived", false)
```

```
      .is("rescheduled_to_appointment_id", null)
      .in("status", ACTIVE_RESERVATION_STATUSES)
      .gte("appointment_date", from)
      .lte("appointment_date", lastAvailableDate),
    supabaseAdmin
      .from("appointment_continuations")
      .select(
        "appointment_id,appointment_date,start_time,assigned_crew_id,booking_d
uration_minutes",
      )
      .gte("appointment_date", from)
      .lte("appointment_date", lastAvailableDate),
    rescheduleHoldsQuery(supabaseAdmin, from, lastAvailableDate),
  ]);

  if (
    blocksRes.error ||
    schedulesRes.error ||
    crewRes.error ||
    exceptionsRes.error ||
    appointmentsRes.error ||
    continuationsRes.error ||
    holdsRes.error
  ) {
    console.error("[Booking continuation] availability lookup failed", {
      blocks: blocksRes.error,
      schedules: schedulesRes.error,
      crew: crewRes.error,
      exceptions: exceptionsRes.error,
      appointments: appointmentsRes.error,
      continuations: continuationsRes.error,
      holds: holdsRes.error,
    });
    return {
      ok: false,
      error: "We could not reserve the next available shop day. Please try again.",
    };
  }

  const continuationParentIds = Array.from(
    new Set((continuationsRes.data ?? []).map((continuation) =>
continuation.appointment_id)),
  );
  const continuationParents = continuationParentIds.length
    ? await supabaseAdmin
```

```
        .from("appointments")
        .select("id,is_archived,status,rescheduled_to_appointment_id")
        .in("id", continuationParentIds)
    : { data: [], error: null };
  if (continuationParents.error) {
    return {
      ok: false,
      error: "We could not reserve the next available shop day. Please try again.",
    };
  }

  const activeContinuationParentIds = new Set(
    (continuationParents.data ?? [])
      .filter(
        (appointment) =>
          !appointment.is_archived &&
          !appointment.rescheduled_to_appointment_id &&
          isActiveReservationStatus(appointment.status),
      )
      .map((appointment) => appointment.id),
  );
  const assignments = [
    ...(appointmentsRes.data ?? []).map((appointment) => ({
      date: appointment.appointment_date,
      startTime: String(appointment.start_time).slice(0, 5),
      durationMinutes: appointment.booking_duration_minutes ?? 60,
      crewId: appointment.assigned_crew_id,
    })),
    ...(continuationsRes.data ?? [])
      .filter((continuation) =>
activeContinuationParentIds.has(continuation.appointment_id))
      .map((continuation) => ({
        date: continuation.appointment_date,
        startTime: String(continuation.start_time).slice(0, 5),
        durationMinutes: continuation.booking_duration_minutes,
        crewId: continuation.assigned_crew_id,
      })),
    ...rescheduleHoldAssignments(holdsRes.data ?? []),
  ];
  const activeCrewIds = new Set((crewRes.data ?? []).map((crew) => crew.id));
  const segments = planContinuationSegmentsFromAvailability({
    source: {
      from,
      to: lastAvailableDate,
      operatingHours,
```

```
      slots: buildBookingTimeSlots(slotConfig, operatingHours),
      blocks: (blocksRes.data ?? []).map((block) => {
        const { endTime } = decodeBlockReason(block.reason);
        return {
          date: block.block_date,
          startTime: block.start_time ? String(block.start_time).slice(0, 5) : null,
          endTime,
          reason: null,
        };
      }),
      assignments,
      schedules: (schedulesRes.data ?? []).filter((schedule) =>
        activeCrewIds.has(schedule.crew_id),
      ),
      exceptions: exceptionsRes.data ?? [],
    },
    firstDate,
    remainingMinutes,
  });
  if (!segments) {
    return {
      ok: false,
      error:
        "There is not enough mechanic availability in the booking window to
complete these services.",
    };
  }

  return {
    ok: true,
    segments: segments.map((segment, index) => ({
      segmentNumber: index + 1,
      appointmentDate: segment.appointmentDate,
      startTime: segment.startTime,
      bookingDurationMinutes: segment.durationMinutes,
      assignedCrewId: segment.crewId,
    })),
  };
}

function availabilityErrorMessage(errors: Array<{ message: string } | null>) {
  const message = errors
    .filter((error): error is { message: string } => !!error)
    .map((error) => error.message)
    .join(" ");
```

```
  if (
    /booking_duration_minutes|appointment_services.*duration_minutes|schedule
_date/i.test(message)
  ) {
    return "Booking capacity setup is incomplete. The shop needs to apply its
scheduling database update.";
  }

  return "We could not load booking availability. Please try again shortly.";
}

function makeReference() {
  const alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
  let out = "";
  const bytes = crypto.getRandomValues(new Uint8Array(6));
  for (const b of bytes) out += alphabet[b % alphabet.length];
  return `FRM-${out}`;
}

function getClientIp() {
  const request = getRequest();
  const platformIp =
    request?.headers.get("x-vercel-forwarded-for")?.trim() ||
    request?.headers.get("cf-connecting-ip")?.trim();
  const forwarded = request?.headers.get("x-forwarded-for")?.split(",")[0]?.trim();
  return platformIp || forwarded || request?.headers.get("x-real-ip")?.trim() ||
"unknown";
}

async function hashRateLimitSubject(subject: string) {
  const digest = await crypto.subtle.digest("SHA-256", new
TextEncoder().encode(subject));
  return Array.from(new Uint8Array(digest), (byte) =>
byte.toString(16).padStart(2, "0")).join("");
}

/**
 * Persistent, per-endpoint limits. The database function serializes increments,
 * so this also works when the app runs across multiple serverless instances.
 */
type RateLimitResult = "allowed" | "limited" | "unavailable";

async function checkPublicRequestRateLimit(
  scope:
```

```
    | "availability"
    | "booking"
    | "lookup"
    | "cancellation"
    | "rescheduling"
    | "verification_request"
    | "verification_verify",
  maxRequests: number,
  windowSeconds: number,
  subject?: string,
): Promise<RateLimitResult> {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const subjects = [await hashRateLimitSubject(`ip:${getClientIp()}`)];
    if (subject) subjects.push(await hashRateLimitSubject(`subject:${subject}`));

    for (const opaqueSubject of subjects) {
      const { data, error } = await supabaseAdmin.rpc("enforce_public_rate_limit", {
        p_scope: scope,
        p_subject: opaqueSubject,
        p_limit: maxRequests,
        p_window_seconds: windowSeconds,
      });
      if (error) {
        console.error(`[Rate limit] ${scope} check failed`, error);
        return "unavailable";
      }
      if (data !== true) return "limited";
    }
    return "allowed";
  } catch (error) {
    console.error(`[Rate limit] ${scope} check failed`, error);
    return "unavailable";
  }
}

async function isTurnstileVerificationValid(token: string, idempotencyKey: string)
{
  const secret = process.env["TURNSTILE_SECRET_KEY"];
  if (!secret) return true;
  if (!token) return false;

  try {
    const body = new FormData();
```

```
    body.set("secret", secret);
    body.set("response", token);
    body.set("idempotency_key", idempotencyKey);
    const ip = getClientIp();
    if (ip !== "unknown") body.set("remoteip", ip);

    const response = await
fetch("https://challenges.cloudflare.com/turnstile/v0/siteverify", {
      method: "POST",
      body,
    });
    const result = (await response.json()) as { success?: unknown; action?:
unknown };
    return response.ok && result.success === true && result.action === "booking";
  } catch (error) {
    console.error("[Turnstile] booking verification failed", error);
    return false;
  }
}

/** Slot availability for a date range (Manila dates). */
export const getAvailability = createServerFn({ method: "GET" })
  .validator((input: unknown) => availabilitySchema.parse(input ?? {}))
  .handler(async ({ data }) => {
    const from = earliestBookableDate();
    const to = addDays(from, data.days);
    const supabaseModule = await
import("@/integrations/supabase/client.server").catch((error) => {
      console.error("[Booking availability] Supabase configuration failed", error);
      return null;
    });
    if (!supabaseModule) {
      return unavailableAvailability(
        from,
        to,
        "Booking availability is temporarily unavailable. Please try again shortly.",
      );
    }
    const { supabaseAdmin } = supabaseModule;
    const bookingRules = await getBookingRules();
    const configuredFrom = firstBookableDate(
      bookingRules,
      data.rescheduling ? bookingRules.reschedulingNoticeHours : undefined,
    );
    const configuredTo = addDays(
```

```
      manilaNow().date,
      Math.max(0, Math.min(data.days, bookingRules.maxAdvanceBookingDays)),
    );

    const availabilityRateLimit = await checkPublicRequestRateLimit("availability",
60, 60);
    if (availabilityRateLimit === "limited") {
      return unavailableAvailability(
        configuredFrom,
        configuredTo,
        "Too many availability checks. Please try again shortly.",
      );
    }
    // Availability is read-only. A rate-limit infrastructure issue must not
    // prevent a customer from selecting a service and seeing schedules.
    if (availabilityRateLimit === "unavailable") {
      console.warn(
        "[Rate limit] availability protection unavailable; continuing with read-only
lookup",
      );
    }

    // Builds fresh queries on each call so a failed attempt can be retried.
    const loadSchedulingData = () => {
      let appointmentsQuery = supabaseAdmin
        .from("appointments")
        .select("appointment_date,start_time,assigned_crew_id,booking_duration_
minutes")
        .eq("is_archived", false)
        .gte("appointment_date", configuredFrom)
        .lte("appointment_date", configuredTo)
        .is("rescheduled_to_appointment_id", null)
        .in("status", ACTIVE_RESERVATION_STATUSES);
      if (data.excludeAppointmentId) {
        appointmentsQuery = appointmentsQuery.neq("id",
data.excludeAppointmentId);
      }
      return Promise.all([
        supabaseAdmin
          .from("time_slots")
          .select("id,start_time,end_time,capacity")
          .eq("is_active", true)
          .order("start_time"),
        supabaseAdmin
          .from("schedule_blocks")
```

```
        .select("block_date,start_time,reason")
        .eq("is_active", true)
        .gte("block_date", configuredFrom)
        .lte("block_date", configuredTo),
      appointmentsQuery,
      data.serviceIds.length > 0
        ? supabaseAdmin
            .from("services")
            .select("id,name,price,duration_minutes,large_bike_price,large_bike_dur
ation_minutes")
            .in("id", data.serviceIds)
            .eq("is_active", true)
            .eq("is_archived", false)
        : {
            data: [] as {
              id: string;
              name: string;
              price: number;
              duration_minutes: number | null;
              large_bike_price: number | null;
              large_bike_duration_minutes: number | null;
            }[],
            error: null,
          },
      supabaseAdmin
        .from("crew_schedules")
        .select("*")
        .gte("schedule_date", configuredFrom)
        .lte("schedule_date", configuredTo),
      supabaseAdmin
        .from("crew_members")
        .select("id")
        .eq("is_active", true)
        .eq("is_archived", false),
      supabaseAdmin
        .from("crew_availability_exceptions")
        .select("*")
        .gte("end_date", configuredFrom)
        .lte("start_date", configuredTo),
      supabaseAdmin
        .from("appointment_continuations")
        .select(
          "appointment_id,appointment_date,start_time,assigned_crew_id,booking
_duration_minutes",
        )
```

```
        .gte("appointment_date", configuredFrom)
        .lte("appointment_date", configuredTo),
      rescheduleHoldsQuery(supabaseAdmin, configuredFrom, configuredTo),
    ]);
  };

  let results = await loadSchedulingData();
  // One retry covers a dropped connection or a brief database restart.
  if (results.some((result) => result.error)) {
    await new Promise((resolve) => setTimeout(resolve, 500));
    results = await loadSchedulingData();
  }
  const [
    slotsRes,
    blocksRes,
    apptsRes,
    servicesRes,
    schedulesRes,
    activeCrewRes,
    exceptionsRes,
    continuationsRes,
    holdsRes,
  ] = results;

  const errors = results.map((result) => result.error);
  if (errors.some(Boolean)) {
    const tables = [
      "time_slots",
      "schedule_blocks",
      "appointments",
      "services",
      "crew_schedules",
      "crew_members",
      "crew_availability_exceptions",
      "appointment_continuations",
      "appointments (reschedule requests)",
    ];
    // Names the project and each failed read, so the hosting logs show the
cause.
    console.error("[Booking availability] database query failed", {
      supabaseProject: supabaseModule.supabaseServerProject(),
      failures: errors.flatMap((error, index) =>
        error
          ? [{ table: tables[index], code: error.code, message: error.message, hint:
error.hint }]
```

```
          : [],
      ),
    });
    return unavailableAvailability(
      configuredFrom,
      configuredTo,
      availabilityErrorMessage(errors),
    );
  }

  if (
    data.serviceIds.length > 0 &&
    (servicesRes.data?.length ?? 0) !== new Set(data.serviceIds).size
  ) {
    return unavailableAvailability(
      configuredFrom,
      configuredTo,
      "One or more selected services are no longer available. Please choose a
different service.",
    );
  }

  let motorcycle: { brand: string; model: string } | undefined;
  if (data.rescheduleReference && data.reschedulePhone) {
    const original = await findAppointment(data.rescheduleReference,
data.reschedulePhone);
    if (!original) {
      return unavailableAvailability(
        configuredFrom,
        configuredTo,
        "We could not verify the original appointment. Please return to the
Appointment Tracker and try again.",
      );
    }

    motorcycle = { brand: original.moto_brand, model: original.moto_model };
  }

  if (!motorcycle && data.motorcycle) {
    motorcycle = { ...data.motorcycle };
  }

  let motorcycleCategory: MotorcycleCcCategory | undefined;
  if (motorcycle) {
    let motorcycleQuery = supabaseAdmin
```